



C++ Training Guide — Chapters 18–21

File I/O · Exception Handling · Namespaces · Smart Pointers

Complete, production-quality examples using `iostream` throughout

18.0	File I/O and File Handling (<code>ofstream</code> · <code>ifstream</code> · <code>fstream</code>)	
19.0	Exception Handling (<code>try</code> · <code>catch</code> · <code>throw</code> · <code>Standard</code>)	
20.0	Namespaces (<code>Defining</code> · <code>Rules</code> · <code>using directive</code>)	
21.0	Smart Pointers (<code>unique_ptr</code> · <code>shared_ptr</code> · <code>weak_ptr</code>)	

18.0 File I/O

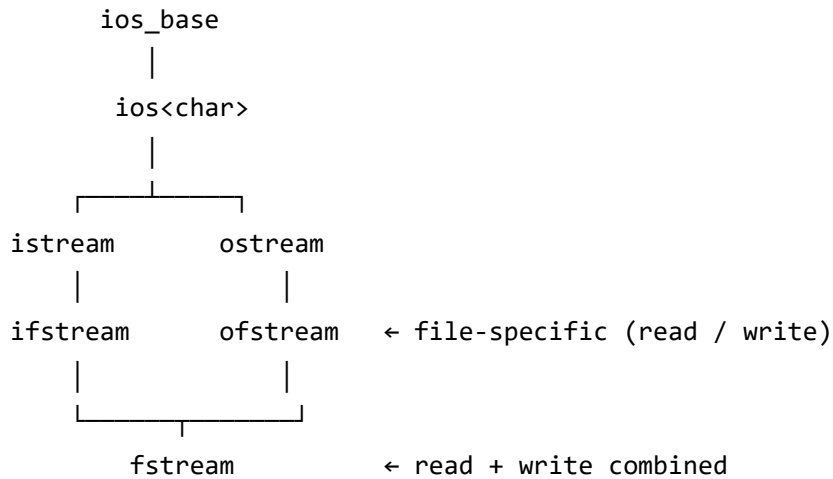
Learning Objectives

- Open, read, write, and close text and binary files using C++ streams
- Distinguish `ofstream` , `ifstream` , and `fstream` and know when to use each
- Use file-position pointers (`seekg` , `seekp` , `tellg` , `tellp`) for random access
- Handle file errors robustly with stream state flags

Introduction: Why File I/O Matters

Every meaningful program needs to persist data beyond its lifetime — configuration files, user records, log files, game saves, binary databases. C++ provides a clean, consistent **stream-based API** that mirrors `std::cout` / `std::cin` , so if you can write to the console, you already know 80% of file I/O.

C++ I/O STREAM INHERITANCE:



Required header: `#include <fstream>`

File Open Modes

Mode flag	Effect
<code>ios::in</code>	Open for reading
<code>ios::out</code>	Open for writing (truncates if exists)
<code>ios::app</code>	Append: always write at end, never truncate
<code>ios::trunc</code>	Truncate file to zero length on open
<code>ios::binary</code>	Binary mode (no newline translation)
<code>ios::ate</code>	Seek to end immediately after open

Modes are combined with `|` :

```
ios::in | ios::out      → read+write (no truncate)
ios::out | ios::binary  → write binary
ios::in | ios::out | ios::binary → read+write binary
```

18.1 File I/O and File Handling

18.2 Streams — ofstream

```
// =====  
// FILE: ofstream_demo.cpp  
// TOPIC: ofstream – Writing to files  
// COMPILE: g++ -std=c++17 -Wall ofstream_demo.cpp -o ofstream_demo  
// =====  
  
#include <iostream>  
#include <fstream>  
#include <string>  
#include <vector>  
#include <iomanip> // setw, setprecision, left, right  
  
// — Data types used throughout —————  
struct Student {  
    std::string name;  
    int         grade;  
    double      gpa;  
    std::string major;  
};  
  
struct Product {  
    int         id;  
    std::string name;  
    double      price;  
    int         stock;  
};  
  
int main() {  
    std::cout << "=== ofstream Demonstration ===\n\n";  
  
    // — 1. Basic text file creation and writing —————  
    {  
        std::cout << "-- 1. Basic write (ios::out | ios::trunc) --\n";  
  
        std::ofstream ofs("hello.txt"); // default: out | trunc
```

```

    if (!ofs.is_open()) {
        std::cerr << "ERROR: Could not open hello.txt for writing.\n";
        return 1;
    }

    ofs << "Hello, File I/O!\n";
    ofs << "This is written using ofstream.\n";
    ofs << "Integer:  " << 42          << "\n";
    ofs << "Double:   " << std::fixed << std::setprecision(4)
        << 3.14159   << "\n";
    ofs << "Boolean:  " << std::boolalpha << true << "\n";

    // File is closed automatically when ofs goes out of scope (RAII).
    // Explicit close is only needed if you want to reuse the object.
    std::cout << "  Written: hello.txt\n\n";
}

// — 2. Writing a formatted CSV table —————
{
    std::cout << "-- 2. Writing CSV (comma-separated) --\n";

    std::vector<Student> students = {
        {"Alice Smith", 92, 3.92, "Computer Science"},
        {"Bob Johnson", 85, 3.50, "Mathematics"},
        {"Carol Williams", 78, 3.10, "Physics"},
        {"David Lee", 96, 4.00, "Computer Science"},
        {"Eve Martinez", 88, 3.75, "Engineering"}
    };

    std::ofstream ofs("students.csv");
    if (!ofs) {
        std::cerr << "ERROR: Cannot create students.csv\n";
        return 1;
    }

    // Header row
    ofs << "Name,Grade,GPA,Major\n";

    for (const auto& s : students) {
        ofs << s.name << ", "
            << s.grade << ", "
            << std::fixed << std::setprecision(2) << s.gpa << ", "
            << s.major << "\n";
    }
}

```

```

    }

    std::cout << "   Written: students.csv ("
                << students.size() << " records)\n\n";
}

// — 3. Writing a formatted fixed-width report —————
{
    std::cout << "-- 3. Fixed-width formatted report --\n";

    std::vector<Product> products = {
        {1001, "Widget Pro",    29.99, 150},
        {1002, "Gadget Ultra",  99.00,  42},
        {1003, "Doohickey",     5.50, 800},
        {1004, "Thingamajig",  199.99,  15},
        {1005, "Whatchamacallit",49.95, 300}
    };

    std::ofstream ofs("products.txt");
    if (!ofs) {
        std::cerr << "ERROR: Cannot create products.txt\n";
        return 1;
    }

    // Title and column headers
    ofs << "=== PRODUCT INVENTORY REPORT ===\n";
    ofs << std::string(55, '-') << "\n";
    ofs << std::left
        << std::setw(6) << "ID"
        << std::setw(22) << "Product Name"
        << std::right
        << std::setw(10) << "Price"
        << std::setw(10) << "Stock"
        << "\n";
    ofs << std::string(55, '-') << "\n";

    double totalValue = 0.0;
    for (const auto& p : products) {
        double lineValue = p.price * p.stock;
        totalValue += lineValue;

        ofs << std::left
            << std::setw(6) << p.id

```

```

        << std::setw(22) << p.name
        << std::right
        << std::setw(9) << std::fixed << std::setprecision(2)
        << p.price << " "
        << std::setw(9) << p.stock
        << "\n";
    }

ofs << std::string(55, '-') << "\n";
ofs << std::right << std::setw(44)
    << "Total inventory value: $"
    << std::setw(11) << std::fixed << std::setprecision(2)
    << totalValue << "\n";

std::cout << "   Written: products.txt\n\n";
}

// — 4. Append mode (ios::app) —————
{
    std::cout << "-- 4. Append mode (ios::app) --\n";

    // Create the log file initially
    {
        std::ofstream ofs("app.log", std::ios::trunc);
        ofs << "[2024-01-01 08:00:00] Application started.\n";
        ofs << "[2024-01-01 08:00:01] Initializing modules.\n";
    }

    // First append session
    {
        std::ofstream ofs("app.log", std::ios::app);
        if (!ofs) { std::cerr << "Cannot open app.log\n"; return 1; }
        ofs << "[2024-01-01 08:00:02] Module A loaded.\n";
        ofs << "[2024-01-01 08:00:03] Module B loaded.\n";
    }

    // Second append session – does NOT erase previous content
    {
        std::ofstream ofs("app.log", std::ios::app);
        if (!ofs) { std::cerr << "Cannot open app.log\n"; return 1; }
        ofs << "[2024-01-01 08:01:00] User logged in: alice\n";
        ofs << "[2024-01-01 08:05:30] Session ended.\n";
    }
}

```

```

    std::cout << "   Written (appended x2): app.log\n\n";
}

// — 5. Explicit open/close to reuse the stream object —————
{
    std::cout << "-- 5. Explicit open() / close() --\n";

    std::ofstream ofs;          // default-constructed (not yet open)

    ofs.open("file_a.txt");
    if (!ofs.is_open()) { std::cerr << "Cannot open file_a.txt\n"; return 1; }
    for (int i = 1; i <= 5; i++)
        ofs << "file_a line " << i << ": " << (i * i) << "\n";
    ofs.close();
    std::cout << "   Written: file_a.txt\n";

    ofs.open("file_b.txt");    // reuse the same ofstream object
    if (!ofs.is_open()) { std::cerr << "Cannot open file_b.txt\n"; return 1; }
    for (int i = 1; i <= 3; i++)
        ofs << "file_b line " << i << ": " << (i * 100) << "\n";
    ofs.close();
    std::cout << "   Written: file_b.txt\n\n";
}

// — 6. Binary write —————
{
    std::cout << "-- 6. Binary mode (ios::binary) --\n";

    struct BinRecord {
        int    id;
        double value;
        char    label[16];
    };

    BinRecord records[3] = {
        {101, 3.14, "pi_approx"},
        {102, 2.718, "euler_e"},
        {103, 1.414, "sqrt2"}
    };

    std::ofstream ofs("data.bin", std::ios::binary | std::ios::trunc);
    if (!ofs) { std::cerr << "Cannot open data.bin\n"; return 1; }
}

```

```

// write() casts struct pointer to char* and writes raw bytes
ofs.write(reinterpret_cast<const char*>(records),
          sizeof(records));

std::cout << "  Written: data.bin ("
          << sizeof(records) << " bytes, "
          << 3 << " records of "
          << sizeof(BinRecord) << " bytes each)\n\n";
}

// — 7. Stream state checking —————
{
    std::cout << "-- 7. Stream state flags --\n";

    std::ofstream ofs("/nonexistent_dir/test.txt"); // will fail

    std::cout << "  is_open(): " << (ofs.is_open() ? "true" : "false") << "\n";
    std::cout << "  good():      " << (ofs.good()    ? "true" : "false") << "\n";
    std::cout << "  fail():      " << (ofs.fail()    ? "true" : "false") << "\n";
    std::cout << "  bad():       " << (ofs.bad()     ? "true" : "false") << "\n";

    // Boolean conversion: if (!ofs) means "something is wrong"
    if (!ofs) {
        std::cout << "  Boolean check: stream is in error state.\n";
    }
}

std::cout << "\n=== ofstream demo complete ===\n";
return 0;
}

```


18.2 Streams — ifstream

```
// =====  
// FILE: ifstream_demo.cpp  
// TOPIC: ifstream – Reading from files  
// NOTE: Run ofstream_demo first to create the files this reads.  
// COMPILE: g++ -std=c++17 -Wall ifstream_demo.cpp -o ifstream_demo  
// =====  
  
#include <iostream>  
#include <fstream>  
#include <sstream>  
#include <string>  
#include <vector>  
#include <iomanip>  
#include <algorithm>  
  
struct Student {  
    std::string name;  
    int         grade;  
    double      gpa;  
    std::string major;  
};  
  
// Parse one CSV line: "Alice Smith,92,3.92,Computer Science"  
Student parseStudentCSV(const std::string& line) {  
    std::istringstream ss(line);  
    Student s;  
    std::getline(ss, s.name, ',');  
    ss >> s.grade;  
    ss.ignore(1, ','); // skip the comma  
    ss >> s.gpa;  
    ss.ignore(1, ',');  
    std::getline(ss, s.major);  
    return s;  
}  
  
int main() {  
    std::cout << "=== ifstream Demonstration ===\n\n";  
  
    // — 1. Read line by line using getline —————  
    {
```

```

std::cout << "-- 1. Read line by line (getline) --\n";

std::ifstream ifs("hello.txt");
if (!ifs.is_open()) {
    std::cerr << "ERROR: hello.txt not found. Run ofstream_demo first.\n";
    return 1;
}

std::string line;
int lineNum = 0;
while (std::getline(ifs, line)) {
    ++lineNum;
    std::cout << std::setw(3) << lineNum << " | " << line << "\n";
}

// Why did the loop end?
if (ifs.eof())
    std::cout << " → Reached end of file after " << lineNum << " lines.\n";
else if (ifs.fail())
    std::cerr << " → Read error occurred!\n";

std::cout << "\n";
}

// — 2. Read word by word using >> —————
{
    std::cout << "-- 2. Read word by word (>>) --\n";

    std::ifstream ifs("hello.txt");
    if (!ifs) { std::cerr << "Cannot open hello.txt\n"; return 1; }

    std::string word;
    int count = 0;
    while (ifs >> word) {
        ++count;
        std::cout << "[" << std::setw(2) << count << "]" " << word << "\n";
    }
    std::cout << " → Total words: " << count << "\n\n";
}

// — 3. Read and parse CSV into structs —————
{
    std::cout << "-- 3. Parse CSV into structs --\n";

```

```

std::ifstream ifs("students.csv");
if (!ifs) { std::cerr << "Cannot open students.csv\n"; return 1; }

std::string line;
std::getline(ifs, line); // skip header row

std::vector<Student> students;
while (std::getline(ifs, line)) {
    if (!line.empty())
        students.push_back(parseStudentCSV(line));
}

// Print formatted table
const int W = 22;
std::cout << std::left
    << std::setw(W) << "Name"
    << std::setw(8) << "Grade"
    << std::setw(7) << "GPA"
    << "Major\n";
std::cout << std::string(60, '-') << "\n";

for (const auto& s : students) {
    std::cout << std::left
        << std::setw(W) << s.name
        << std::setw(8) << s.grade
        << std::setw(7) << std::fixed
        << std::setprecision(2) << s.gpa
        << s.major << "\n";
}

// Sort by GPA descending and print top student
auto top = *std::max_element(students.begin(), students.end(),
    [](const Student& a, const Student& b){ return a.gpa < b.gpa; });

std::cout << "\n → Top student: "
    << top.name << " (GPA: " << top.gpa << ")\n\n";
}

// — 4. Read entire file into a single string (file slurp) —————
{
    std::cout << "-- 4. Slurp entire file into string --\n";

```

```

std::ifstream ifs("app.log");
if (!ifs) { std::cerr << "Cannot open app.log\n"; return 1; }

// istreambuf_iterator reads raw characters (faster than getline loop)
std::string content(
    (std::istreambuf_iterator<char>(ifs)),
    std::istreambuf_iterator<char>())
);

std::cout << "  File size: " << content.size() << " characters\n";
std::cout << "  Contents:\n";
std::cout << content;
std::cout << "\n";
}

// — 5. Read and process formatted data with >> —————
{
    std::cout << "-- 5. Reading mixed-type formatted data --\n";

    // Write a temp file with typed data
    {
        std::ofstream ofs("typed_data.txt");
        ofs << "Alice 25 87500.50\n";
        ofs << "Bob   32 95000.00\n";
        ofs << "Carol 28 72000.75\n";
    }

    std::ifstream ifs("typed_data.txt");
    if (!ifs) { std::cerr << "Cannot open typed_data.txt\n"; return 1; }

    std::string name;
    int         age;
    double      salary;

    std::cout << std::left
        << std::setw(10) << "Name"
        << std::setw(8)  << "Age"
        << "Salary\n";
    std::cout << std::string(30, '-') << "\n";

    while (ifs >> name >> age >> salary) {
        std::cout << std::left
            << std::setw(10) << name

```

```

        << std::setw(8) << age
        << "$" << std::fixed << std::setprecision(2)
        << salary << "\n";
    }
    std::cout << "\n";
}

// — 6. Binary read – reconstruct structs from data.bin —————
{
    std::cout << "-- 6. Binary read --\n";

    struct BinRecord {
        int    id;
        double value;
        char    label[16];
    };

    std::ifstream ifs("data.bin", std::ios::binary);
    if (!ifs) { std::cerr << "Cannot open data.bin\n"; return 1; }

    BinRecord rec;
    int count = 0;
    while (ifs.read(reinterpret_cast<char*>(&rec), sizeof(BinRecord))) {
        ++count;
        std::cout << "  Record " << count
                  << ": id=" << rec.id
                  << "  value=" << std::fixed << std::setprecision(3)
                  << rec.value
                  << "  label=\"" << rec.label << "\"\n";
    }

    std::streamsize lastRead = ifs.gcount();
    std::cout << "  → " << count << " full records read. "
              << "Last gcount()=" << lastRead << "\n\n";
}

// — 7. Checking stream state mid-read —————
{
    std::cout << "-- 7. Stream state mid-read --\n";

    std::ifstream ifs("hello.txt");
    if (!ifs) { std::cerr << "Cannot open hello.txt\n"; return 1; }
}

```

```

auto showState = [&](const std::string& ctx) {
    std::cout << " [" << ctx << "]"
        << " good=" << ifs.good()
        << " eof=" << ifs.eof()
        << " fail=" << ifs.fail()
        << " bad=" << ifs.bad() << "\n";
};

showState("after open");

std::string word;
while (ifs >> word) {} // exhaust the stream
showState("after reading to EOF");

ifs.clear(); // reset eofbit/failbit
ifs.seekg(0); // rewind
showState("after clear() + seekg(0)");

std::string line;
if (std::getline(ifs, line))
    std::cout << " First line (re-read): " << line << "\n";
}

std::cout << "\n=== ifstream demo complete ===\n";
return 0;
}

```

18.2 Streams — fstream

```
// =====  
// FILE: fstream_demo.cpp  
// TOPIC: fstream – Simultaneous read and write  
// COMPILE: g++ -std=c++17 -Wall fstream_demo.cpp -o fstream_demo  
// =====  
  
#include <iostream>  
#include <fstream>  
#include <sstream>  
#include <string>  
#include <vector>  
#include <iomanip>  
#include <algorithm>  
  
int main() {  
    std::cout << "=== fstream Demonstration ===\n\n";  
  
    // — 1. Create file then read it back (open/close cycle) —————  
    {  
        std::cout << "-- 1. Create then read back --\n";  
  
        std::fstream fs;  
  
        // Phase 1: write  
        fs.open("notes.txt", std::ios::out | std::ios::trunc);  
        if (!fs) { std::cerr << "Cannot create notes.txt\n"; return 1; }  
  
        fs << "Note 001: fstream allows read and write on one object.\n";  
        fs << "Note 002: Must close and reopen (or seekg) to switch modes.\n";  
        fs << "Note 003: Always check is_open() before using.\n";  
        fs << "Note 004: RAII closes the file automatically.\n";  
        fs.close();    // must close before reopening in different mode  
  
        // Phase 2: read back  
        fs.open("notes.txt", std::ios::in);  
        if (!fs) { std::cerr << "Cannot read notes.txt\n"; return 1; }  
  
        std::cout << "  notes.txt content:\n";  
        std::string line;  
        while (std::getline(fs, line))
```

```

        std::cout << "    " << line << "\n";

    std::cout << "\n";
}

// — 2. Open for read+write simultaneously (no truncate) —————
{
    std::cout << "-- 2. Read+Write simultaneously (ios::in | ios::out) --\n";

    // First create a file
    {
        std::ofstream ofs("counter.txt", std::ios::trunc);
        ofs << "visits: 0\n";
    }

    // Open r+w: read the count, increment it, write it back
    std::fstream fs("counter.txt", std::ios::in | std::ios::out);
    if (!fs) { std::cerr << "Cannot open counter.txt\n"; return 1; }

    // Read current count
    std::string label;
    int count;
    fs >> label >> count;
    std::cout << "  Read: " << label << " " << count << "\n";

    // Update it
    count++;

    // Rewrite the whole file (simplest approach for small files)
    fs.seekp(0); // put pointer to beginning
    fs << "visits: " << count << "\n";
    fs.flush();

    // Read it back to verify
    fs.seekg(0);
    std::string line;
    std::getline(fs, line);
    std::cout << "  After update: " << line << "\n\n";
}

// — 3. In-place update with fixed-width text records —————
{
    std::cout << "-- 3. In-place fixed-width record update --\n";

```



```

// Fixed-width record format: 15 bytes each
// Format: "Name      ScoreNN" (10-char name, 2-char score, newline)
// Exact layout: name(10) score(3) '\n' = 14 chars = 14-byte record

// Create initial data
{
    std::ofstream ofs("scores.txt", std::ios::trunc);
    ofs << "Alice      085\n"; // 14 bytes per record
    ofs << "Bob        072\n";
    ofs << "Carol      091\n";
    ofs << "David      063\n";
    ofs << "Eve        088\n";
}

const std::streamoff RECORD_LEN = 14;

// Read all records
auto readAll = [&]() {
    std::ifstream ifs("scores.txt");
    std::cout << "  Current scores:\n";
    std::string line;
    int idx = 0;
    while (std::getline(ifs, line))
        std::cout << "    [" << idx++ << "] " << line << "\n";
};

readAll();

// Update Bob (index 1) – change score from 072 to 099
std::fstream fs("scores.txt", std::ios::in | std::ios::out);
if (!fs) { std::cerr << "Cannot open scores.txt\n"; return 1; }

// Jump write-pointer to record 1, column for score (offset 10)
fs.seekp(1 * RECORD_LEN + 10);
fs << "099";
fs.flush();
fs.close();

std::cout << "\n  After updating Bob's score to 099:\n";
readAll();
std::cout << "\n";
}

```

```
// — 4. Counting and modifying lines in-place (string substitution)-
{
    std::cout << "-- 4. Line-based search and replace (load→modify→save) --\n";

    // Write source file
    {
        std::ofstream ofs("template.txt");
        ofs << "Dear {{NAME}},\n";
        ofs << "Your order {{ORDER_ID}} has been shipped.\n";
        ofs << "Expected delivery: {{DATE}}.\n";
        ofs << "Thank you, {{NAME}}!\n";
    }

    // Read into memory, substitute, write back
    std::vector<std::string> lines;
    {
        std::ifstream ifs("template.txt");
        std::string line;
        while (std::getline(ifs, line))
            lines.push_back(line);
    }

    // Substitution map (in real code use std::map)
    struct KV { std::string key, value; };
    std::vector<KV> replacements = {
        {"{{NAME}}", "Alice Smith"},
        {"{{ORDER_ID}}", "ORD-20240115"},
        {"{{DATE}}", "2024-01-22"}
    };

    auto substitute = [&](std::string& s) {
        for (const auto& kv : replacements) {
            size_t pos = 0;
            while ((pos = s.find(kv.key, pos)) != std::string::npos) {
                s.replace(pos, kv.key.size(), kv.value);
                pos += kv.value.size();
            }
        }
    };

    {
        std::ofstream ofs("letter.txt");
```

```

        for (auto& line : lines) {
            substitute(line);
            ofs << line << "\n";
        }
    }

    // Print result
    std::cout << "  Generated letter.txt:\n";
    std::ifstream ifs("letter.txt");
    std::string line;
    while (std::getline(ifs, line))
        std::cout << "    " << line << "\n";
    std::cout << "\n";
}

// — 5. All four stream state flags explained —————
{
    std::cout << "-- 5. Stream state flags --\n";

    std::fstream fs("hello.txt", std::ios::in);
    if (!fs) { std::cerr << "Cannot open hello.txt\n"; return 1; }

    auto showState = [&](const std::string& label) {
        std::cout << "  [" << label << "]\n"
            << "    good()=" << fs.good() << "  (no errors at all)\n"
            << "    eof() =" << fs.eof() << "  (end of file reached)\n"
            << "    fail()=" << fs.fail() << "  (logical error / parse fail)\n"
            << "    bad() =" << fs.bad() << "  (hardware I/O error)\n";
    };

    showState("just opened");

    std::string w;
    while (fs >> w) {} // consume until EOF
    showState("after reading to EOF");

    fs.clear(); // clears eof+fail bits
    fs.seekg(0);
    showState("after clear() and seekg(0)");

    if (std::getline(fs, w))
        std::cout << "    Re-read first line: " << w << "\n";
}

```

```
std::cout << "\n=== fstream demo complete ===\n";  
return 0;  
}
```

18.3 File Position Pointers

```
// =====  
// FILE: file_position_demo.cpp  
// TOPIC: seekg, seekp, tellg, tellp – random file access  
// COMPILE: g++ -std=c++17 -Wall file_position_demo.cpp -o file_position_demo  
// =====  
  
#include <iostream>  
#include <fstream>  
#include <string>  
#include <vector>  
#include <iomanip>  
#include <cstring>      // memset  
  
// -----  
// A fixed-size binary record suitable for random access  
// -----  
struct Contact {  
    int id;           // 4 bytes  
    char firstName[20]; // 20 bytes  
    char lastName[20]; // 20 bytes  
    char email[40];    // 40 bytes  
    char phone[15];    // 15 bytes  
    int age;           // 4 bytes  
    // -----  
    // 103 bytes total (check: sizeof(Contact))  
};  
  
// Helper: create a Contact safely, zero-filling char arrays  
Contact makeContact(int id, const std::string& fn, const std::string& ln,  
                    const std::string& email, const std::string& phone, int age) {  
    Contact c{}; // zero-initializes all fields  
    c.id = id;  
    c.age = age;  
    fn.copy(c.firstName, sizeof(c.firstName) - 1);  
    ln.copy(c.lastName, sizeof(c.lastName) - 1);  
    email.copy(c.email, sizeof(c.email) - 1);  
    phone.copy(c.phone, sizeof(c.phone) - 1);  
    return c;  
}
```

```

// Helper: print one Contact record
void printContact(const Contact& c) {
    std::cout << std::setw(5) << c.id
        << " " << std::left
        << std::setw(20) << c.firstName
        << std::setw(20) << c.lastName
        << std::setw(30) << c.email
        << std::setw(16) << c.phone
        << "age=" << c.age << "\n";
}

// Helper: print all records in the file sequentially
void printAllContacts(const std::string& filename) {
    std::ifstream ifs(filename, std::ios::binary);
    if (!ifs) { std::cerr << "Cannot open " << filename << "\n"; return; }

    std::cout << std::right
        << std::setw(5) << "ID"
        << " " << std::left
        << std::setw(20) << "First Name"
        << std::setw(20) << "Last Name"
        << std::setw(30) << "Email"
        << std::setw(16) << "Phone"
        << "Age\n";
    std::cout << std::string(100, '-') << "\n";

    Contact c;
    while (ifs.read(reinterpret_cast<char*>(&c), sizeof(Contact))) {
        printContact(c);
    }
    std::cout << "\n";
}

const std::string DB_FILE = "contacts.bin";

int main() {
    std::cout << "=== File Position Pointers Demo ===\n";
    std::cout << "    sizeof(Contact) = " << sizeof(Contact) << " bytes\n\n";

    // — 1. Build the binary database —————
    {
        std::cout << "-- 1. Building binary contacts database --\n";
    }
}

```

```

std::vector<Contact> contacts = {
    makeContact(1001,"Alice", "Smith", "alice@example.com", "555-0101", 28),
    makeContact(1002,"Bob", "Johnson", "bob@example.com", "555-0102", 34),
    makeContact(1003,"Carol", "Williams", "carol@example.com", "555-0103", 22),
    makeContact(1004,"David", "Lee", "david@example.com", "555-0104", 45),
    makeContact(1005,"Eve", "Martinez", "eve@example.com", "555-0105", 31),
    makeContact(1006,"Frank", "Brown", "frank@example.com", "555-0106", 29),
    makeContact(1007,"Grace", "Taylor", "grace@example.com", "555-0107", 37),
};

std::ofstream ofs(DB_FILE, std::ios::binary | std::ios::trunc);
if (!ofs) { std::cerr << "Cannot create " << DB_FILE << "\n"; return 1; }

for (const auto& c : contacts)
    ofs.write(reinterpret_cast<const char*>(&c), sizeof(Contact));

std::cout << " Created " << DB_FILE << " with "
    << contacts.size() << " records.\n";
std::cout << " File size = "
    << contacts.size() * sizeof(Contact) << " bytes\n\n";
}

// — 2. tellg – discover file size without reading content —————
{
    std::cout << "-- 2. tellg: measure file size --\n";

    // ios::ate: seek to end on open so tellg() gives file size
    std::ifstream ifs(DB_FILE, std::ios::binary | std::ios::ate);
    if (!ifs) { std::cerr << "Cannot open " << DB_FILE << "\n"; return 1; }

    std::streampos fileSize = ifs.tellg();
    int numRecords = static_cast<int>(fileSize) / sizeof(Contact);

    std::cout << " tellg() immediately after open (ios::ate) = "
        << fileSize << " bytes\n";
    std::cout << " Number of records = " << numRecords << "\n\n";
}

// — 3. Sequential read – verify the database —————
{
    std::cout << "-- 3. Sequential read (tellg mid-stream) --\n";

    std::ifstream ifs(DB_FILE, std::ios::binary);

```

```

if (!ifs) { std::cerr << "Cannot open " << DB_FILE << "\n"; return 1; }

Contact c;
int index = 0;

std::cout << std::right << std::setw(5) << "ID"
          << " " << std::left
          << std::setw(10) << "First"
          << std::setw(10) << "Last"
          << std::setw(10) << "Position-after-read\n";
std::cout << std::string(40, '-') << "\n";

while (ifs.read(reinterpret_cast<char*>(&c), sizeof(Contact))) {
    std::cout << std::right << std::setw(5) << c.id
              << " " << std::left
              << std::setw(10) << c.firstName
              << std::setw(10) << c.lastName
              << "tellg()=" << ifs.tellg() << "\n";
    ++index;
}
std::cout << " → Read " << index << " records.\n\n";
}

// — 4. seekg(offset, ios::beg) — jump to any record directly —
{
    std::cout << "-- 4. seekg: direct random access (ios::beg) --\n";

    std::ifstream ifs(DB_FILE, std::ios::binary);
    if (!ifs) { std::cerr << "Cannot open " << DB_FILE << "\n"; return 1; }

    // Read record at index 4 (Eve, 0-based)
    int targetIndex = 4;
    std::streamoff offset = static_cast<std::streamoff>(targetIndex)
                          * sizeof(Contact);

    ifs.seekg(offset, std::ios::beg);
    std::cout << " seekg(" << offset << ", beg) → tellg()="
              << ifs.tellg() << "\n";

    Contact c;
    ifs.read(reinterpret_cast<char*>(&c), sizeof(Contact));

    std::cout << " Record [" << targetIndex << "]: ";

```



```

    printContact(c);
    std::cout << "\n";
}

// — 5. seekg(offset, ios::end) – read last record —————
{
    std::cout << "-- 5. seekg from end (ios::end) --\n";

    std::ifstream ifs(DB_FILE, std::ios::binary);
    if (!ifs) { std::cerr << "Cannot open " << DB_FILE << "\n"; return 1; }

    // Negative offset from end = go back one record
    std::streamoff oneRecord = static_cast<std::streamoff>(sizeof(Contact));
    ifs.seekg(-oneRecord, std::ios::end);
    std::cout << "    seekg(-" << oneRecord << ", end) → tellg()="
                << ifs.tellg() << "\n";

    Contact c;
    ifs.read(reinterpret_cast<char*>(&c), sizeof(Contact));
    std::cout << "    Last record: ";
    printContact(c);
    std::cout << "\n";
}

// — 6. seekg(offset, ios::cur) – skip records —————
{
    std::cout << "-- 6. seekg relative to current pos (ios::cur) --\n";

    std::ifstream ifs(DB_FILE, std::ios::binary);
    if (!ifs) { std::cerr << "Cannot open " << DB_FILE << "\n"; return 1; }

    Contact c;

    // Read record 0 (Alice)
    ifs.read(reinterpret_cast<char*>(&c), sizeof(Contact));
    std::cout << "    Read [0]: " << c.firstName << " " << c.lastName << "\n";

    // Skip record 1 (Bob)
    ifs.seekg(sizeof(Contact), std::ios::cur);
    std::cout << "    Skipped [1] using seekg(sizeof(Contact), cur)\n";

    // Read record 2 (Carol)
    ifs.read(reinterpret_cast<char*>(&c), sizeof(Contact));

```

```

std::cout << "  Read [2]: " << c.firstName << " " << c.lastName << "\n";

// Seek backward – go back to record 1
ifs.seekg(-2 * static_cast<std::streamoff>(sizeof(Contact)),
        std::ios::cur);
ifs.read(reinterpret_cast<char*>(&c), sizeof(Contact));
std::cout << "  Searched back, read [1]: "
        << c.firstName << " " << c.lastName << "\n\n";
}

// — 7. seekp – in-place update (modify Bob's email and age) ———
{
    std::cout << "-- 7. seekp: in-place binary record update --\n";

    std::fstream fs(DB_FILE, std::ios::binary | std::ios::in | std::ios::out);
    if (!fs) { std::cerr << "Cannot open " << DB_FILE << "\n"; return 1; }

    // Step 1: read Bob's record (index 1) via seekg
    int bobIndex = 1;
    std::streamoff bobOffset = static_cast<std::streamoff>(bobIndex
        * sizeof(Contact));

    fs.seekg(bobOffset, std::ios::beg);
    Contact bob;
    fs.read(reinterpret_cast<char*>(&bob), sizeof(Contact));

    std::cout << "  Bob BEFORE: ";
    printContact(bob);

    // Step 2: modify in memory
    std::string newEmail = "bob.updated@newdomain.com";
    std::memset(bob.email, 0, sizeof(bob.email));
    newEmail.copy(bob.email, sizeof(bob.email) - 1);
    bob.age = 35;

    // Step 3: write back – use seekp to the same offset
    fs.seekp(bobOffset, std::ios::beg);
    fs.write(reinterpret_cast<const char*>(&bob), sizeof(Contact));
    fs.flush();
    fs.close();

    // Verify
    std::cout << "\n  Bob AFTER update:\n";
}

```

```

std::ifstream verify(DB_FILE, std::ios::binary);
verify.seekg(bobOffset, std::ios::beg);
Contact updated;
verify.read(reinterpret_cast<char*>(&updated), sizeof(Contact));
printContact(updated);
std::cout << "\n";
}

// — 8. tellp – track write position —————
{
    std::cout << "-- 8. tellp: tracking write position --\n";

    std::ofstream ofs("position_log.txt");
    if (!ofs) { std::cerr << "Cannot create position_log.txt\n"; return 1; }

    std::vector<std::string> entries = {
        "Entry A: short\n",
        "Entry B: this one is longer than A\n",
        "Entry C: medium length line\n"
    };

    std::cout << " Byte positions after each write:\n";
    for (const auto& e : entries) {
        ofs << e;
        std::cout << "    After writing \"" << e.substr(0, 10) << "...\"";
        << " tellp()=" << ofs.tellp() << "\n";
    }
    std::cout << "\n";
}

// — 9. Full database after all updates —————
{
    std::cout << "-- 9. Final database state --\n";
    printAllContacts(DB_FILE);
}

// — 10. Practical: append a new record to binary file —————
{
    std::cout << "-- 10. Append a new record --\n";

    // Open in append+binary: seekp goes to end automatically
    std::fstream fs(DB_FILE,
        std::ios::binary | std::ios::in | std::ios::out | std::ios::app);

```

```

    if (!fs) { std::cerr << "Cannot open " << DB_FILE << "\n"; return 1; }

    Contact newContact = makeContact(
        1008, "Henry", "Wilson", "henry@example.com", "555-0108", 26);

    fs.write(reinterpret_cast<const char*>(&newContact), sizeof(Contact));
    fs.flush();
    std::cout << "    Appended: Henry Wilson (ID=1008)\n\n";

    std::cout << "    Updated database:\n";
}

printAllContacts(DB_FILE);

std::cout << "=== File Position Pointers demo complete ===\n";
return 0;
}

```

19.0 Exception Handling

Learning Objectives

- Understand why exceptions are superior to error-code returns
- Use `try`, `catch`, and `throw` with full confidence
- Build a custom exception class hierarchy
- Work with all major standard exceptions
- Handle uncaught exceptions using `std::set_terminate`

Introduction: The Problem with Error Codes

ERROR CODE STYLE (C):

```
int r1 = openDB();
if(r1 < 0) return handleError();
int r2 = loadUser(id);
if(r2 < 0) return handleError();
int r3 = processUser();
if(r3 < 0) return handleError();
int r4 = saveResult();
if(r4 < 0) return handleError();
```

Error handling is interleaved
with every single operation.

EXCEPTION STYLE (C++):

```
try {
    openDB();
    loadUser(id);
    processUser();
    saveResult();
}
catch(const DBException& e) {
    cerr << e.what();
}
```

Business logic is clean.
Errors handled in one place.

19.1 Basics / 19.2 try, catch, throw / 19.3 Standard Exceptions / 19.4 Uncaught

Exceptions

```
// =====  
// FILE: exception_demo.cpp  
// TOPIC: Complete exception handling demonstration  
// COMPILE: g++ -std=c++17 -Wall exception_demo.cpp -o exception_demo  
// =====  
  
#include <iostream>  
#include <fstream>  
#include <stdexcept>  
#include <exception>  
#include <string>  
#include <vector>  
#include <limits>  
#include <cmath>  
#include <new>  
#include <typeinfo>  
#include <functional>  
#include <sstream>  
  
// =====  
// SECTION A: CUSTOM EXCEPTION CLASS HIERARCHY  
// =====  
  
// Base for all application exceptions  
class AppException : public std::exception {  
    std::string message;  
    std::string source;    // which function threw  
    int         errorCode;  
public:  
    AppException(const std::string& msg,  
                 const std::string& src = "unknown",  
                 int         code = 0)  
        : message(msg), source(src), errorCode(code) {}  
  
    const char* what() const noexcept override {  
        return message.c_str();  
    }  
    const std::string& getSource() const { return source; }  
    int               getErrorCode() const { return errorCode; }
```

```

    virtual std::string fullDescription() const {
        std::ostringstream oss;
        oss << "[Code " << errorCode << "]" "
            << "(" << source << ")" "
            << message;
        return oss.str();
    }
};

// — Mid-level exceptions —————
class DatabaseException : public AppException {
    std::string query;
public:
    DatabaseException(const std::string& msg,
                     const std::string& q    = "",
                     int                code = 2000)
        : AppException(msg, "DatabaseLayer", code), query(q) {}

    std::string fullDescription() const override {
        return AppException::fullDescription()
            + (query.empty() ? "" : "\n Query: " + query);
    }
};

class NetworkException : public AppException {
    std::string endpoint;
    int         httpStatus;
public:
    NetworkException(const std::string& msg,
                     const std::string& ep    = "",
                     int                http  = 0,
                     int                code  = 3000)
        : AppException(msg, "NetworkLayer", code),
          endpoint(ep), httpStatus(http) {}

    std::string fullDescription() const override {
        std::ostringstream oss;
        oss << AppException::fullDescription();
        if (!endpoint.empty()) oss << "\n Endpoint: " << endpoint;
        if (httpStatus)       oss << "\n HTTP: "    << httpStatus;
        return oss.str();
    }
};

```



```

class ValidationException : public AppException {
    std::string field;
    std::string receivedValue;
public:
    ValidationException(const std::string& fld,
                       const std::string& reason,
                       const std::string& received = "")
        : AppException("Validation failed for '" + fld + "': " + reason,
                       "ValidationLayer", 4000),
          field(fld), receivedValue(received) {}

    const std::string& getField() const { return field; }

    std::string fullDescription() const override {
        std::ostringstream oss;
        oss << AppException::fullDescription();
        oss << "\n Field: " << field;
        if (!receivedValue.empty())
            oss << "\n Received: '" << receivedValue << "'";
        return oss.str();
    }
};

class FileException : public AppException {
    std::string filename;
public:
    FileException(const std::string& msg, const std::string& fname)
        : AppException(msg, "FileLayer", 5000), filename(fname) {}

    std::string fullDescription() const override {
        return AppException::fullDescription()
            + "\n File: " + filename;
    }
};

// =====
// SECTION B: FUNCTIONS THAT THROW
// =====

int safeDivide(int numerator, int denominator) {
    if (denominator == 0)
        throw std::invalid_argument("Division by zero: denominator cannot be 0");
}

```

```

    return numerator / denominator;
}

double safeSqrt(double x) {
    if (x < 0.0)
        throw std::domain_error("sqrt of negative number: " + std::to_string(x));
    return std::sqrt(x);
}

void validateUserInput(const std::string& username,
                      int age,
                      const std::string& email) {
    if (username.empty())
        throw ValidationException("username", "must not be empty", username);

    if (username.length() < 3)
        throw ValidationException("username",
            "must be at least 3 characters (got " +
            std::to_string(username.length()) + ")", username);

    if (age < 0 || age > 130)
        throw ValidationException("age",
            "must be between 0 and 130",
            std::to_string(age));

    if (email.find('@') == std::string::npos)
        throw ValidationException("email",
            "must contain '@' character", email);

    std::cout << "  Input valid: username='" << username
        << "' age=" << age
        << " email=" << email << "\n";
}

int queryDatabase(const std::string& sql) {
    if (sql.empty())
        throw DatabaseException("Empty SQL query", sql, 2001);
    if (sql.find("DROP") != std::string::npos ||
        sql.find("TRUNCATE") != std::string::npos)
        throw DatabaseException("Dangerous DDL statement rejected", sql, 2002);
    if (sql.length() > 500)
        throw DatabaseException("Query too long (max 500 chars)", sql, 2003);
}

```

```

    // Simulate success
    return 42;    // rows affected
}

std::string fetchFromNetwork(const std::string& url, int timeoutMs) {
    if (url.empty())
        throw NetworkException("URL must not be empty", url, 0, 3001);
    if (url.substr(0, 4) != "http")
        throw NetworkException("Only HTTP(S) URLs supported", url, 0, 3002);
    if (timeoutMs <= 0)
        throw NetworkException("Timeout must be positive ms",
                                url, 0, 3003);

    // Simulate a "404" scenario
    if (url.find("missing") != std::string::npos)
        throw NetworkException("Resource not found", url, 404, 3004);

    return "{\"status\": \"ok\", \"data\": \"sample\"}";
}

void openConfig(const std::string& filename) {
    std::ifstream ifs(filename);
    if (!ifs.is_open())
        throw FileException("Cannot open configuration file", filename);
    // If open: read contents (omitted here)
    std::cout << "  Config file '" << filename << "' opened OK.\n";
}

// Re-throw demonstration
void processUserRequest(const std::string& query) {
    try {
        int rows = queryDatabase(query);
        std::cout << "  DB returned " << rows << " rows.\n";
    } catch (const DatabaseException& e) {
        std::cerr << "  [processUserRequest] Caught DB exception, re-throwing.\n";
        throw;    // re-throw: preserves the original exception type exactly
    }
}

// =====
// SECTION C: EXCEPTION SAFETY PATTERNS
// =====

```

```

// Strong exception guarantee via copy-and-swap
class Config {
    std::vector<std::pair<std::string, std::string>> entries;
public:
    void addEntry(const std::string& key, const std::string& value) {
        // No exceptions possible here → basic guarantee is fine
        entries.emplace_back(key, value);
    }

    // Strong guarantee: either ALL of newEntries are added, or NONE
    void replaceAll(std::vector<std::pair<std::string, std::string>> newEntries) {
        // Work on a temporary copy
        std::vector<std::pair<std::string, std::string>> temp = std::move(newEntries);

        // Validate all before committing (could throw)
        for (const auto& [k, v] : temp) {
            if (k.empty())
                throw ValidationException("config_key", "key must not be empty");
        }

        // If we reach here – validation passed. Commit atomically.
        entries.swap(temp); // noexcept: can't fail
    }

    void print() const {
        for (const auto& [k, v] : entries)
            std::cout << "    " << k << " = " << v << "\n";
    }

    int size() const { return static_cast<int>(entries.size()); }
};

// RAII guard: rolls back a resource if scope exits via exception
class TransactionGuard {
    bool committed;
    std::string name;
public:
    TransactionGuard(const std::string& n) : committed(false), name(n) {
        std::cout << " [" << name << "] Transaction BEGIN\n";
    }

    void commit() {
        committed = true;
        std::cout << " [" << name << "] Transaction COMMITTED\n";
    }
};

```

```

    }

    ~TransactionGuard() {
        if (!committed)
            std::cout << " [" << name << "] Transaction ROLLED BACK\n";
    }
};

// =====
// SECTION D: STANDARD EXCEPTION TYPES
// =====

void demonstrateStandardExceptions() {
    std::cout << "\n=== Standard Exception Types ===\n\n";

    // Helper lambda – run a function and catch std::exception
    auto tryRun = [](const std::string& label,
                    std::function<void()> fn) {
        std::cout << "-- " << label << " --\n";
        try { fn(); }
        catch (const std::exception& e) {
            std::cout << " Caught [" << typeid(e).name()
                        << "]: " << e.what() << "\n\n";
        }
    };

    // std::out_of_range
    tryRun("std::out_of_range", [](){
        std::vector<int> v = {10, 20, 30};
        std::cout << " v.at(10) = " << v.at(10) << "\n"; // throws
    });

    // std::invalid_argument
    tryRun("std::invalid_argument (stoi)", [](){
        std::string s = "not_a_number";
        int n = std::stoi(s); // throws
        (void)n;
    });

    // std::length_error
    tryRun("std::length_error", [](){
        std::string s;
        s.resize(s.max_size() + 1); // throws
    });
}

```

```

});

// std::domain_error
tryRun("std::domain_error (sqrt)", [](){
    double result = safeSqrt(-9.0); // throws via our wrapper
    (void)result;
});

// std::bad_alloc
tryRun("std::bad_alloc", [](){
    auto* p = new double[std::numeric_limits<std::size_t>::max() / 2];
    (void)p;
});

// std::bad_cast
tryRun("std::bad_cast", [](){
    struct Base    { virtual ~Base() = default; };
    struct Derived : Base {};
    struct Other   : Base {};

    Derived d;
    Base& b = d;
    Other& o = dynamic_cast<Other&>(b); // bad_cast: b is Derived, not Other
    (void)o;
});

// std::runtime_error
tryRun("std::runtime_error", [](){
    throw std::runtime_error("External service unavailable (HTTP 503)");
});

// std::overflow_error
tryRun("std::overflow_error", [](){
    throw std::overflow_error("Integer overflow in factorial calculation");
});

// std::range_error
tryRun("std::range_error", [](){
    throw std::range_error("Result exceeds representable double range");
});

// Catching by base class
std::cout << "-- Catching by base std::exception --\n";

```

```

std::vector<std::function<void()>> operations = {
    []{ std::vector<int>{}.at(5); },
    []{ throw std::runtime_error("timeout"); },
    []{ throw std::overflow_error("too big"); },
};

for (auto& op : operations) {
    try { op(); }
    catch (const std::exception& e) {
        std::cout << " Caught via base: " << e.what() << "\n";
    }
}
std::cout << "\n";
}

// =====
// SECTION E: UNCAUGHT EXCEPTIONS
// =====

void demonstrateUncaughtHandling() {
    std::cout << "=== Uncaught Exception Handling ===\n\n";

    // — Custom terminate handler —————
    std::set_terminate([]() {
        std::cerr << "\n[FATAL] std::terminate called!\n";
        try {
            // Try to get info about the current exception
            std::rethrow_exception(std::current_exception());
        } catch (const std::exception& e) {
            std::cerr << "[FATAL] type: " << typeid(e).name() << "\n";
            std::cerr << "[FATAL] what: " << e.what() << "\n";
        } catch (...) {
            std::cerr << "[FATAL] Non-std exception (type unknown)\n";
        }
        std::cerr << "[FATAL] Performing emergency cleanup...\n";
        std::abort();
    });

    std::cout << "-- 1. noexcept function --\n";
    // noexcept: if this function throws, terminate is called immediately
    auto noexceptFunc = []() noexcept -> int {
        // Can't throw – handle errors defensively
        return 42;
    };

```

```

};
std::cout << " noexcept result: " << noExceptFunc() << "\n\n";

// — 2. Stack unwinding – RAII destructors always called —————
std::cout << "-- 2. Stack unwinding – RAII in action --\n";
{
    class CleanupGuard {
        std::string resource;
    public:
        CleanupGuard(const std::string& r) : resource(r) {
            std::cout << " Acquired: " << resource << "\n";
        }
        ~CleanupGuard() {
            std::cout << " Released: " << resource << "\n";
        }
    };

    try {
        CleanupGuard g1("DatabaseConnection");
        CleanupGuard g2("NetworkSocket");
        CleanupGuard g3("FileHandle");
        std::cout << " Throwing inside guarded scope...\n";
        throw std::runtime_error("Simulated mid-operation failure");
        // All three guards' destructors fire during stack unwind
    } catch (const std::runtime_error& e) {
        std::cout << " Caught: " << e.what() << "\n";
        std::cout << " All resources properly released by RAII.\n\n";
    }
}

// — 3. exception_ptr – transport exceptions across call stacks ———
std::cout << "-- 3. exception_ptr (cross-context transport) --\n";
{
    std::exception_ptr capturedEx = nullptr;

    // Simulate work that might fail (e.g., in a thread or callback)
    auto doWork = [&]() {
        try {
            throw NetworkException("Connection refused",
                                   "http://api.example.com", 503, 3099);
        } catch (...) {
            capturedEx = std::current_exception(); // capture it
            std::cout << " Worker: exception captured.\n";
        }
    };
}

```



```

    }
};

doWork();

// Later, re-throw it in the original context
if (capturedEx) {
    std::cout << "  Main: re-throwing captured exception.\n";
    try {
        std::rethrow_exception(capturedEx);
    } catch (const NetworkException& e) {
        std::cout << "  Main caught NetworkException:\n"
                  << "    " << e.fullDescription() << "\n\n";
    }
}
}

// — 4. Exception in constructor — no leak —————
std::cout << "-- 4. Exception in constructor (no leak) --\n";
{
    class StrictPositiveArray {
        int* data;
        int size;
    public:
        StrictPositiveArray(int sz) : data(nullptr), size(sz) {
            if (sz <= 0)
                throw std::invalid_argument(
                    "Array size must be positive, got: " +
                    std::to_string(sz));
            data = new int[sz]();
            std::cout << "  Allocated array of " << sz << " ints.\n";
        }
        ~StrictPositiveArray() {
            delete[] data;
            std::cout << "  Array freed.\n";
        }
        int& operator[](int i) { return data[i]; }
    };

    // Constructor throws → no object created → no destructor needed
    try {
        StrictPositiveArray bad(-5);
    } catch (const std::invalid_argument& e) {

```

```

        std::cout << " Constructor threw: " << e.what() << "\n";
    }

    // Successful construction
    try {
        StrictPositiveArray good(4);
        good[0] = 100; good[1] = 200;
        std::cout << " good[0]=" << good[0] << " good[1]=" << good[1] << "\n";
    } catch (...) {
        std::cout << " Unexpected exception!\n";
    }
    std::cout << "\n";
}

}

// =====
// MAIN
// =====

int main() {
    std::cout << "=== Complete Exception Handling Demo ===\n\n";

    // — Part 1: Basic throw and catch —————
    std::cout << "=== Part 1: Basic throw/catch ===\n\n";

    std::cout << "-- safeDivide --\n";
    try {
        std::cout << " 10/2 = " << safeDivide(10, 2) << "\n";
        std::cout << " 10/0 = " << safeDivide(10, 0) << "\n";
    } catch (const std::invalid_argument& e) {
        std::cout << " Caught: " << e.what() << "\n";
    }
    std::cout << " (execution continues here)\n\n";

    std::cout << "-- safeSqrt --\n";
    for (double v : {16.0, -4.0, 0.0}) {
        try {
            std::cout << " sqrt(" << v << ") = " << safeSqrt(v) << "\n";
        } catch (const std::domain_error& e) {
            std::cout << " Domain error: " << e.what() << "\n";
        }
    }
    std::cout << "\n";
}

```

// — Part 2: Multiple catch blocks —————

```
std::cout << "=== Part 2: Multiple catch blocks ===\n\n";
```

```
auto runValidation = [](const std::string& u, int a, const std::string& e) {
    std::cout << "   Validating: user='" << u
                << "' age=" << a << " email='" << e << "'\n";
    try {
        validateUserInput(u, a, e);
    } catch (const ValidationException& ve) {
        std::cout << "   Validation error:\n"
                  << "       " << ve.fullDescription() << "\n";
    } catch (const AppException& ae) {
        std::cout << "   App error: " << ae.what() << "\n";
    } catch (const std::exception& se) {
        std::cout << "   Std error: " << se.what() << "\n";
    } catch (...) {
        std::cout << "   Unknown error!\n";
    }
};
```

```
runValidation("Alice", 28, "alice@example.com"); // valid
runValidation("", 28, "alice@example.com"); // empty name
runValidation("Bo", 28, "bob@example.com"); // name too short
runValidation("Carol", -5, "carol@example.com"); // bad age
runValidation("Dave", 30, "dave_at_example.com"); // bad email
std::cout << "\n";
```

// — Part 3: Database exceptions —————

```
std::cout << "=== Part 3: Database exceptions ===\n\n";
```

```
auto runQuery = [](const std::string& sql) {
    std::cout << "   Query: '" << sql << "'\n";
    try {
        int rows = queryDatabase(sql);
        std::cout << "   OK: " << rows << " rows\n";
    } catch (const DatabaseException& e) {
        std::cout << "   DB Exception:\n       "
                  << e.fullDescription() << "\n";
    }
};
```

```
runQuery("SELECT * FROM users WHERE active = 1");
```

```

runQuery("");
runQuery("DROP TABLE users");
std::cout << "\n";

// — Part 4: Network exceptions —————
std::cout << "=== Part 4: Network exceptions ===\n\n";

auto fetchURL = [](const std::string& url, int timeout) {
    std::cout << "  Fetching: " << url << "\n";
    try {
        std::string result = fetchFromNetwork(url, timeout);
        std::cout << "  Response: " << result << "\n";
    } catch (const NetworkException& e) {
        std::cout << "  Network Exception:\n      "
            << e.fullDescription() << "\n";
    }
};

fetchURL("http://api.example.com/data", 5000);
fetchURL("http://api.example.com/missing/resource", 5000);
fetchURL("ftp://old-server.com/file", 5000);
fetchURL("", 5000);
std::cout << "\n";

// — Part 5: File exceptions —————
std::cout << "=== Part 5: File exceptions ===\n\n";

// Create a real config file, then try a missing one
{ std::ofstream ofs("app.cfg"); ofs << "debug=true\ntimeout=30\n"; }

for (const auto& fname : {"app.cfg", "missing.cfg"}) {
    std::cout << "  Opening: " << fname << "\n";
    try {
        openConfig(fname);
    } catch (const FileException& e) {
        std::cout << "  File Exception:\n      " << e.fullDescription() << "\n";
    }
}
std::cout << "\n";

// — Part 6: Re-throw —————
std::cout << "=== Part 6: Re-throw ===\n\n";
try {

```

```

        processUserRequest("DROP TABLE accounts");
    } catch (const DatabaseException& e) {
        std::cout << " [main] Caught re-thrown exception:\n    "
                  << e.fullDescription() << "\n\n";
    }

// — Part 7: Exception safety – RAII transaction —————
std::cout << "=== Part 7: RAII Transaction Guard ===\n\n";

std::cout << "-- Successful transaction --\n";
try {
    TransactionGuard txn("Transfer-001");
    queryDatabase("UPDATE accounts SET balance = balance - 100 WHERE id=1");
    queryDatabase("UPDATE accounts SET balance = balance + 100 WHERE id=2");
    txn.commit();
} catch (const std::exception& e) {
    std::cout << " Error: " << e.what() << "\n";
}

std::cout << "\n-- Failed transaction (auto rollback) --\n";
try {
    TransactionGuard txn("Transfer-002");
    queryDatabase("UPDATE accounts SET balance = balance - 500 WHERE id=1");
    queryDatabase("DROP TABLE accounts"); // throws → txn not committed
    txn.commit();
} catch (const DatabaseException& e) {
    std::cout << " Error: " << e.what() << "\n";
    // TransactionGuard destructor fires → prints ROLLED BACK
}
std::cout << "\n";

// — Part 8: Config strong guarantee —————
std::cout << "=== Part 8: Strong exception guarantee ===\n\n";

Config cfg;
cfg.addEntry("host", "localhost");
cfg.addEntry("port", "5432");
std::cout << " Config before:\n"; cfg.print();

// Valid replacement
try {
    cfg.replaceAll({"host", "db.prod.com"}, {"port", "5433"}, {"pool", "10"});
    std::cout << " Config after valid replaceAll:\n"; cfg.print();
}

```

```

} catch (const ValidationException& e) {
    std::cout << "  replaceAll rejected: " << e.what() << "\n";
    std::cout << "  Config unchanged:\n"; cfg.print();
}

// Invalid replacement (empty key) – config must remain unchanged
std::cout << "\n  Trying invalid replaceAll (empty key)...\n";
try {
    cfg.replaceAll({"host","new.host"}, {"","bad_key"}, {"debug","true"}));
    std::cout << "  Config after (should NOT reach here):\n"; cfg.print();
} catch (const ValidationException& e) {
    std::cout << "  replaceAll rejected: " << e.what() << "\n";
    std::cout << "  Config unchanged (strong guarantee):\n"; cfg.print();
}
std::cout << "\n";

// — Part 9: catch(...) – catch anything —————
std::cout << "=== Part 9: catch(...) ===\n\n";
try {
    throw 99;    // throwing a plain int (not an exception object – unusual)
} catch (const std::exception& e) {
    std::cout << "  std::exception: " << e.what() << "\n";
} catch (int i) {
    std::cout << "  Caught int: " << i << "\n";
} catch (...) {
    std::cout << "  Caught unknown type.\n";
}

// — Part 10: Standard exceptions and uncaught handling —————
demonstrateStandardExceptions();
demonstrateUncaughtHandling();

std::cout << "=== Exception Handling demo complete ===\n";
return 0;
}

```

20.0 Namespaces

Learning Objectives

- Understand why name conflicts occur and how namespaces prevent them
- Define, reopen, and nest namespaces
- Use namespace aliases, `using` declarations, and `using` directives correctly
- Understand anonymous namespaces and ADL (Argument Dependent Lookup)

20.1 Understanding Namespaces / 20.2

Rules of Namespaces

```
// =====  
// FILE: namespace_demo.cpp  
// TOPIC: Complete namespace demonstration  
// COMPILE: g++ -std=c++17 -Wall namespace_demo.cpp -o namespace_demo  
// =====  
  
#include <iostream>  
#include <string>  
#include <vector>  
#include <sstream>  
#include <iomanip>  
#include <algorithm>  
#include <cmath>  
  
// =====  
// RULE 1: Basic namespace definition  
// Everything inside belongs to that named scope.  
// =====  
  
namespace geometry {  
  
    const double PI = 3.14159265358979323846;  
  
    struct Point {  
        double x, y;  
  
        Point(double x = 0.0, double y = 0.0) : x(x), y(y) {}  
  
        double distanceTo(const Point& other) const {  
            double dx = x - other.x;  
            double dy = y - other.y;  
            return std::sqrt(dx * dx + dy * dy);  
        }  
  
        std::string str() const {  
            std::ostringstream oss;  
            oss << std::fixed << std::setprecision(2)  
                << "(" << x << ", " << y << ")";  
            return oss.str();  
        }  
    }  
}
```

```

    Point operator+(const Point& o) const { return {x+o.x, y+o.y}; }
    Point operator-(const Point& o) const { return {x-o.x, y-o.y}; }
    bool operator==(const Point& o) const {
        return std::abs(x-o.x) < 1e-9 && std::abs(y-o.y) < 1e-9;
    }
};

class Circle {
    Point center;
    double radius;
public:
    Circle(Point c, double r) : center(c), radius(r) {
        if (r <= 0) throw std::invalid_argument("Radius must be positive");
    }
    double area() const { return PI * radius * radius; }
    double circumference() const { return 2.0 * PI * radius; }
    bool contains(const Point& p) const {
        return center.distanceTo(p) <= radius;
    }
    std::string str() const {
        std::ostringstream oss;
        oss << std::fixed << std::setprecision(2)
            << "Circle{center=" << center.str()
            << ", r=" << radius << "}";
        return oss.str();
    }
};

```

```

// Free function inside namespace
double triangleArea(const Point& a, const Point& b, const Point& c) {
    // Shoelace / cross-product formula
    return std::abs((b.x - a.x) * (c.y - a.y) -
        (c.x - a.x) * (b.y - a.y)) / 2.0;
}

```

```

double distanceBetween(const Point& a, const Point& b) {
    return a.distanceTo(b);
}

```

```

} // namespace geometry

```

```

// =====

```

```
// RULE 2: A namespace can be REOPENED (additive across translation units)
```

```
// =====
```

```
namespace geometry { // reopening – adds Rectangle to the same namespace
```

```
class Rectangle {
    Point origin; // top-left corner
    double width, height;
public:
    Rectangle(Point o, double w, double h)
        : origin(o), width(w), height(h) {}

    double area() const { return width * height; }
    double perimeter() const { return 2.0 * (width + height); }
    bool contains(const Point& p) const {
        return p.x >= origin.x && p.x <= origin.x + width &&
            p.y >= origin.y && p.y <= origin.y + height;
    }
    std::string str() const {
        std::ostringstream oss;
        oss << std::fixed << std::setprecision(2)
            << "Rect{origin=" << origin.str()
            << ", " << width << "x" << height << "}";
        return oss.str();
    }
};

class Triangle {
    Point a, b, c;
public:
    Triangle(Point a, Point b, Point c) : a(a), b(b), c(c) {}
    double area() const { return triangleArea(a, b, c); }
    double perimeter() const {
        return a.distanceTo(b) + b.distanceTo(c) + c.distanceTo(a);
    }
    std::string str() const {
        return "Triangle{" + a.str() + ", " + b.str() + ", " + c.str() + "}";
    }
};
```

```
} // namespace geometry (second opening)
```

```
// =====
```

```
// RULE 3: NESTED namespaces (namespaces within namespaces)
```

```
//
```

```
namespace app {
```

```
    // C++17 shorthand: namespace app::config { ... }
```

```
    // For clarity we write them explicitly here:
```

```
namespace config {
```

```
    const std::string APP_NAME    = "GeometryApp";
```

```
    const std::string VERSION    = "2.1.0";
```

```
    const int        MAX_SHAPES  = 10000;
```

```
    const double     EPSILON     = 1e-9;
```

```
    struct AppConfig {
```

```
        std::string logLevel    = "INFO";
```

```
        bool        debugMode   = false;
```

```
        int         threadCount = 4;
```

```
        std::string str() const {
```

```
            std::ostringstream oss;
```

```
            oss << "AppConfig{log=" << logLevel
```

```
                << " debug=" << std::boolalpha << debugMode
```

```
                << " threads=" << threadCount << "}";
```

```
            return oss.str();
```

```
        }
```

```
    };
```

```
} // namespace config
```

```
namespace logging {
```

```
    enum class Level { TRACE, DEBUG, INFO, WARNING, ERROR, FATAL };
```

```
    std::string levelStr(Level l) {
```

```
        switch (l) {
```

```
            case Level::TRACE:    return "TRACE";
```

```
            case Level::DEBUG:    return "DEBUG";
```

```
            case Level::INFO:     return "INFO";
```

```
            case Level::WARNING:  return "WARNING";
```

```
            case Level::ERROR:    return "ERROR";
```

```
            case Level::FATAL:    return "FATAL";
```

```

        default:                return "UNKNOWN";
    }
}

class Logger {
    std::string name;
    Level      minLevel;
    std::vector<std::string> history;    // stores recent messages
public:
    Logger(const std::string& n, Level min = Level::INFO)
        : name(n), minLevel(min) {}

    void log(Level lvl, const std::string& msg) {
        if (lvl >= minLevel) {
            std::string entry = "[" + levelStr(lvl) + "][" + name + "] " + msg;
            std::cout << entry << "\n";
            history.push_back(entry);
        }
    }

    void trace (const std::string& m) { log(Level::TRACE,  m); }
    void debug  (const std::string& m) { log(Level::DEBUG,  m); }
    void info   (const std::string& m) { log(Level::INFO,   m); }
    void warning(const std::string& m) { log(Level::WARNING, m); }
    void error  (const std::string& m) { log(Level::ERROR,  m); }
    void fatal  (const std::string& m) { log(Level::FATAL,  m); }

    const std::vector<std::string>& getHistory() const { return history; }
    int historySize() const { return static_cast<int>(history.size()); }
};

} // namespace logging

namespace math {

    // Demonstrates that geometry::PI and app::math have no conflict
    const double TAU = 2.0 * geometry::PI;    // using another namespace

    double degreesToRadians(double deg) { return deg * geometry::PI / 180.0; }
    double radiansToDegrees(double rad) { return rad * 180.0 / geometry::PI; }

    bool approxEqual(double a, double b, double eps = geometry::PI * 1e-10) {
        return std::abs(a - b) < eps;
    }
}

```

```

    }

} // namespace math

namespace services {

    class ShapeService {
    app::logging::Logger logger;
    std::vector<std::string> shapeLog;
    public:
        ShapeService() : logger("ShapeService", app::logging::Level::DEBUG) {}

        void addCircle(const geometry::Point& center, double radius) {
            geometry::Circle c(center, radius);
            std::string entry = "Added " + c.str() +
                " area=" + std::to_string(c.area());
            logger.info(entry);
            shapeLog.push_back(entry);
        }

        void addRectangle(const geometry::Point& origin,
            double w, double h) {
            geometry::Rectangle r(origin, w, h);
            std::string entry = "Added " + r.str() +
                " area=" + std::to_string(r.area());
            logger.info(entry);
            shapeLog.push_back(entry);
        }

        void printSummary() const {
            std::cout << " ShapeService: " << shapeLog.size()
                << " shapes registered.\n";
            for (const auto& s : shapeLog)
                std::cout << " " << s << "\n";
        }
    };

} // namespace services

} // namespace app

// =====
// RULE 4: Anonymous namespace – file-private scope

```

```

// Acts like 'static' at file level. Not accessible from other files.
// =====

namespace {
    // Only this translation unit can see these:
    int    g_callCount    = 0;
    double g_totalWork    = 0.0;

    void recordWork(double amount) {
        ++g_callCount;
        g_totalWork += amount;
    }

    std::string formatMetrics() {
        std::ostringstream oss;
        oss << std::fixed << std::setprecision(2)
            << "calls=" << g_callCount
            << " totalWork=" << g_totalWork;
        return oss.str();
    }
} // anonymous namespace

// =====
// RULE 5: Namespace ALIAS – short name for long namespace
// =====

namespace geo    = geometry;           // 'geo' is an alias for 'geometry'
namespace applog = app::logging;        // short alias for nested namespace
namespace appcfg = app::config;
namespace appmath= app::math;

// =====
// RULE 6: using DECLARATION – import one specific name
// =====

using geometry::Point;                 // only Point is imported; Circle is not
using geometry::PI;                    // bring PI directly into scope

// =====
// RULE 7: ADL – Argument Dependent Lookup
// The compiler searches the namespace of argument types for functions.
// =====

```

```

namespace mylib {
    struct Tensor {
        int rows, cols;
        Tensor(int r, int c) : rows(r), cols(c) {}
    };

    // Free function in mylib
    void describe(const Tensor& t) {
        std::cout << "  mylib::describe → Tensor(" << t.rows
                    << "x" << t.cols << ")\n";
    }

    Tensor transpose(const Tensor& t) {
        return Tensor(t.cols, t.rows);
    }
}

// =====
//  RULE 8: Inline namespace – default version selection (C++11)
//  =====

namespace api {
    inline namespace v2 {                // v2 is the "current" version
        void greet() {
            std::cout << "  api::greet() → api::v2::greet() (inline)\n";
        }
        int compute(int x) { return x * x; }
    }

    namespace v1 {                      // old version still accessible explicitly
        void greet() {
            std::cout << "  api::v1::greet() (legacy)\n";
        }
        int compute(int x) { return x + x; }
    }
}

// =====
//  MAIN
//  =====

int main() {
    std::cout << "=== Namespace Complete Demo ===\n\n";
}

```



```
// — 1. Fully qualified access —————
std::cout << "=== 1. Fully qualified names ===\n";
geometry::Point p1(0.0, 0.0);
geometry::Point p2(3.0, 4.0);
geometry::Circle circ(p1, 5.0);
geometry::Rectangle rect(geometry::Point(1.0,1.0), 6.0, 4.0);
geometry::Triangle tri (geometry::Point(0,0),
                        geometry::Point(4,0),
                        geometry::Point(0,3));

std::cout << "  p1 = " << p1.str() << "\n";
std::cout << "  p2 = " << p2.str() << "\n";
std::cout << "  p2-p1 distance: " << p1.distanceTo(p2) << "\n";
std::cout << "  PI = " << geometry::PI << "\n";
std::cout << "  " << circ.str()
          << " area=" << std::fixed << std::setprecision(3) << circ.area() << "\n";
std::cout << "  " << rect.str()
          << " area=" << rect.area() << "\n";
std::cout << "  " << tri.str()
          << " area=" << tri.area() << "\n";
std::cout << "  circ contains p2? "
          << (circ.contains(p2) ? "YES" : "NO") << "\n\n";

// — 2. Namespace alias (geo = geometry) —————
std::cout << "=== 2. Namespace alias (geo = geometry) ===\n";
geo::Point p3(1.0, 1.0), p4(5.0, 4.0);
std::cout << "  Triangle area p1,p3,p4: "
          << std::fixed << std::setprecision(3)
          << geo::triangleArea(p1, p3, p4) << "\n";
std::cout << "  geo::PI = " << geo::PI << "\n\n";

// — 3. using declaration (Point, PI imported at file scope) —————
std::cout << "=== 3. using declaration (Point and PI) ===\n";
Point p5(7.0, 8.0); // No geometry:: prefix – using declaration
std::cout << "  p5 = " << p5.str() << "\n";
std::cout << "  PI = " << PI << " (no geometry:: prefix)\n\n";

// — 4. Nested namespaces —————
std::cout << "=== 4. Nested namespaces ===\n";
std::cout << "  App: " << appcfg::APP_NAME
          << " v" << appcfg::VERSION << "\n";
std::cout << "  Max shapes: " << appcfg::MAX_SHAPES << "\n";
```

```

appcfg::AppConfig cfg;
cfg.debugMode    = true;
cfg.logLevel     = "DEBUG";
cfg.threadCount  = 8;
std::cout << " " << cfg.str() << "\n\n";

// — 5. Logging namespace —————
std::cout << "=== 5. Logging namespace ===\n";
applog::Logger logger("Demo", applog::Level::TRACE);
logger.trace  ("Entering main function");
logger.debug  ("Config loaded successfully");
logger.info   ("Application started");
logger.warning("Memory usage above 80%");
logger.error  ("Database connection failed (will retry)");
std::cout << "  Logger history size: " << logger.historySize() << "\n\n";

// — 6. Math namespace —————
std::cout << "=== 6. Math namespace ===\n";
std::cout << "  TAU = " << std::fixed << std::setprecision(6)
           << appmath::TAU << "\n";
std::cout << "  90° in radians: "
           << appmath::degreesToRadians(90.0) << "\n";
std::cout << "  π radians in degrees: "
           << appmath::radiansToDegrees(geometry::PI) << "\n";
std::cout << "  approxEqual(0.1+0.2, 0.3): "
           << std::boolalpha
           << appmath::approxEqual(0.1+0.2, 0.3) << "\n\n";

// — 7. Services namespace —————
std::cout << "=== 7. Services namespace ===\n";
app::services::ShapeService shapeSvc;
shapeSvc.addCircle  (geometry::Point(0,0), 5.0);
shapeSvc.addCircle  (geometry::Point(3,4), 2.0);
shapeSvc.addRectangle(geometry::Point(1,1), 8.0, 5.0);
std::cout << "\n";
shapeSvc.printSummary();
std::cout << "\n";

// — 8. Anonymous namespace —————
std::cout << "=== 8. Anonymous namespace (file-private) ===\n";
recordWork(3.14);
recordWork(2.72);

```

```

recordWork(1.41);
std::cout << "  Metrics: " << formatMetrics() << "\n\n";

// — 9. ADL —————
std::cout << "=== 9. Argument Dependent Lookup (ADL) ===\n";
mylib::Tensor t1(3, 4);
describe(t1);           // found via ADL: no 'mylib::' needed!
mylib::Tensor t2 = transpose(t1); // also ADL
std::cout << "  Transposed: " << t2.rows << "x" << t2.cols << "\n\n";

// — 10. Inline namespace —————
std::cout << "=== 10. Inline namespace ===\n";
api::greet();           // calls api::v2::greet (inline = default)
api::v2::greet();       // explicit
api::v1::greet();       // explicit legacy
std::cout << "  api::compute(5) = " << api::compute(5) << " (v2: 5*5)\n";
std::cout << "  api::v1::compute(5) = "
           << api::v1::compute(5) << " (v1: 5+5)\n\n";

// — 11. using directive (local scope) —————
std::cout << "=== 11. using directive (local scope only) ===\n";
{
    using namespace geometry;
    // Now Circle, Rectangle, Triangle, Point, PI all available unqualified
    Circle c2(Point(0,0), 4.0);
    std::cout << "  (inside block) " << c2.str()
              << " area=" << std::fixed << std::setprecision(2)
              << c2.area() << "\n";
    std::cout << "  (inside block) PI = " << PI << "\n";
}
// Outside block: geometry:: required again (no leakage)
// geometry::Circle c3(...); ← needs prefix here

// — 12. Alias declarations —————
std::cout << "\n=== 12. Alias declarations ===\n";
using PointVec = std::vector<geometry::Point>; // type alias
using LogLevel = app::logging::Level;         // alias to enum

PointVec pts = {
    geometry::Point(0,0),
    geometry::Point(1,0),
    geometry::Point(0,1)
};

```

```
std::cout << "   PointVec has " << pts.size() << " points\n";
std::cout << "   LogLevel::WARNING via alias: "
           << app::logging::levelStr(LogLevel::WARNING) << "\n\n";
```

```
// — 13. Scope and conflict summary
std::cout << "=== 13. Scope rules summary ===\n";
std::cout << R"(
```

NAMESPACE RULE SUMMARY:

Access method	Visibility
ns::name	Always precise. No ambiguity.
using ns::name	One name imported to this scope
namespace alias = ns	Short synonym for long ns path
using namespace ns	ALL names from ns. Use locally!
namespace { }	File-private (translation unit)
inline namespace	Default version selection
ADL (no qualifier)	Auto from argument type's ns

KEY RULES:

- ✓ Namespaces are OPEN: you can add to them from multiple files.
- ✓ Anonymous namespace = internal linkage (C++ replacement for 'static').
- ✓ 'using namespace std' in headers POLLUTES every file that includes it.
- ✓ Prefer 'using ns::name' over 'using namespace ns' in large scopes.
- ✓ Namespace aliases fix the "too long to type" problem cleanly.

```
);
```

```
std::cout << "=== Namespace demo complete ===\n";
return 0;
```

```
}
```

21.0 Smart Pointers

Learning Objectives

- Understand why raw pointers cause memory leaks, double frees, and dangling accesses
- Use `unique_ptr` for exclusive ownership
- Use `shared_ptr` for shared ownership with reference counting

- Use `weak_ptr` to observe shared objects without owning them
- Know the performance characteristics and trade-offs of each

Introduction: The Raw Pointer Problem

THREE DEADLY RAW POINTER BUGS:

```
// 1. MEMORY LEAK – forget to delete
void leak() {
    int* p = new int(42);
    if (something) return;    // forgot delete! Leak!
    delete p;
}
```

```
// 2. DANGLING POINTER – use after delete
int* p = new int(5);
delete p;
*p = 10;    // UNDEFINED BEHAVIOR – p is dangling
```

```
// 3. DOUBLE FREE – delete twice
int* p = new int(3);
delete p;
delete p;    // CRASH or heap corruption
```

SMART POINTERS eliminate all three automatically via RAII.

21.1 Introduction / 21.2 Types of Smart

Pointers

```
// =====  
// FILE: smart_pointers_demo.cpp  
// TOPIC: auto_ptr (history), unique_ptr, shared_ptr, weak_ptr  
// COMPILE: g++ -std=c++17 -Wall smart_pointers_demo.cpp -o smart_ptrs  
// =====  
  
#include <iostream>  
#include <memory>  
#include <string>  
#include <vector>  
#include <utility>      // std::move  
#include <functional>    // std::function  
#include <algorithm>  
  
// =====  
// TRACKED RESOURCE – reveals construction and destruction clearly  
// =====  
  
class Resource {  
    std::string name;  
    int         id;  
    double      value;  
    static int  nextId;  
  
public:  
    explicit Resource(const std::string& n, double v = 0.0)  
        : name(n), id(++nextId), value(v) {  
        std::cout << "  [+] Resource #" << id  
                    << " \"" << name << "\" created.\n";  
    }  
  
    ~Resource() {  
        std::cout << "  [-] Resource #" << id  
                    << " \"" << name << "\" destroyed.\n";  
    }  
  
    void use() const {  
        std::cout << "  [*] Resource #" << id  
                    << " \"" << name << "\" in use (value=" << value << ")\n";  
    }  
}
```

```

    void setValue(double v) { value = v; }
    double getValue() const { return value; }
    int getId() const { return id; }
    const std::string& getName() const { return name; }
};

int Resource::nextId = 0;

// =====
// auto_ptr – DEPRECATED (C++11) and REMOVED (C++17)
// Shown for historical understanding only.
// =====

void autoPtrHistory() {
    std::cout << "\n=== auto_ptr (Deprecated – History Only) ===\n\n";

    std::cout << "  auto_ptr was C++'s first smart pointer (C++98).\n";
    std::cout << "  It had a FATAL FLAW: copying an auto_ptr MOVED ownership\n";
    std::cout << "  silently – the source became null without any warning.\n\n";

    std::cout << R"(  // This was LEGAL but DANGEROUS in C++03:
auto_ptr<Resource> a(new Resource("Old"));
auto_ptr<Resource> b = a;  // 'a' is NOW null – silent ownership move!
a->use();  // CRASH: a is null – no compiler warning!

// This broke std::vector:
vector<auto_ptr<Resource>> v;
v.push_back(auto_ptr<Resource>(new Resource("Item")));
// Internally, vector copies elements – auto_ptr copy = data loss!
)";

    std::cout << "\n  REPLACEMENT:\n";
    std::cout << "    unique_ptr → explicit move (std::move required)\n";
    std::cout << "    shared_ptr → shared ownership with ref count\n";
    std::cout << "    auto_ptr is removed in C++17. Never use it.\n\n";
}

// =====
// unique_ptr – Exclusive, non-copyable ownership
// =====

void uniquePtrDemo() {

```



```

std::cout << "=== unique_ptr: Exclusive Ownership ===\n\n";

// — 1. Creation with make_unique (preferred over 'new') —————
std::cout << "-- 1. make_unique (preferred) --\n";
{
    // make_unique<T>(constructor args) – exception safe, no bare 'new'
    auto up1 = std::make_unique<Resource>("Alpha", 1.0);
    up1->use();
    std::cout << "  up1 is " << (up1 ? "not null" : "null") << "\n";
    // Resource destroyed here by unique_ptr destructor
}
std::cout << "\n";

// — 2. Move semantics – explicit ownership transfer —————
std::cout << "-- 2. std::move – explicit ownership transfer --\n";
{
    auto up1 = std::make_unique<Resource>("Beta", 2.0);
    std::cout << "  Before move: up1=" << (up1 ? "has owner" : "null") << "\n";

    // MUST use std::move – cannot copy a unique_ptr
    auto up2 = std::move(up1);
    std::cout << "  After  move: up1=" << (up1 ? "has owner" : "null")
                << "  up2=" << (up2 ? "has owner" : "null") << "\n";

    up2->use();    // Beta is alive and owned by up2

    // auto up3 = up1;    ← COMPILE ERROR: unique_ptr is non-copyable
}
std::cout << "\n";

// — 3. unique_ptr in std::vector —————
std::cout << "-- 3. unique_ptr in vector --\n";
{
    std::vector<std::unique_ptr<Resource>> pool;

    for (int i = 0; i < 4; i++) {
        // push_back with std::move (or emplace_back)
        pool.push_back(
            std::make_unique<Resource>("Pool-" + std::to_string(i+1),
                                       static_cast<double>(i+1) * 10.0));
    }

    std::cout << "  Using all resources:\n";
}

```

```

    for (const auto& r : pool) r->use();

    // Erase Pool-2 (index 1) – destructor fires immediately
    std::cout << "\n Erasing index 1:\n";
    pool.erase(pool.begin() + 1);

    std::cout << "\n Remaining:\n";
    for (const auto& r : pool) r->use();
    // All remaining destroyed when pool goes out of scope
}
std::cout << "\n";

// — 4. release() – give up ownership without deleting —————
std::cout << "-- 4. release() – raw pointer, no deletion --\n";
{
    auto up = std::make_unique<Resource>("Gamma", 3.0);
    up->use();

    // release(): unique_ptr gives up ownership, returns raw pointer
    Resource* raw = up.release();
    std::cout << " After release(): up is " << (up ? "not null" : "null") << "\n";
    std::cout << " raw is still valid: "; raw->use();
    delete raw; // YOU must delete after release()!
    std::cout << " raw manually deleted.\n";
}
std::cout << "\n";

// — 5. reset() – delete current, optionally own new one —————
std::cout << "-- 5. reset() --\n";
{
    auto up = std::make_unique<Resource>("Delta", 4.0);
    up->use();

    up.reset(); // Delta destroyed here
    std::cout << " After reset(): up is " << (up ? "not null" : "null") << "\n";

    up.reset(new Resource("Epsilon", 5.0)); // now owns Epsilon
    up->use();
    // Epsilon destroyed when up goes out of scope
}
std::cout << "\n";

// — 6. Passing unique_ptr to functions —————

```

```

std::cout << "-- 6. Passing unique_ptr to functions --\n";
{
    // By const reference: observe without owning
    auto observe = [](const std::unique_ptr<Resource>& r) {
        std::cout << " (observe, caller keeps ownership) "; r->use();
    };

    // By raw pointer/reference: use without ownership semantics
    auto use_raw = [](Resource& r) {
        std::cout << " (by ref, no ownership) "; r.use();
    };

    // By value (move): callee takes ownership
    auto consume = [](std::unique_ptr<Resource> r) {
        std::cout << " (consume, callee now owns) "; r->use();
        // r destroyed here
    };

    auto r = std::make_unique<Resource>("Zeta", 6.0);

    observe(r);           // r still owns Zeta
    use_raw(*r);           // r still owns Zeta
    consume(std::move(r)); // r gives up Zeta; Zeta destroyed in consume
    std::cout << " After consume: r is " << (r ? "not null" : "null") << "\n";
}
std::cout << "\n";

// — 7. Custom deleter —————
std::cout << "-- 7. Custom deleter --\n";
{
    // Lambda deleter: logs before deleting
    auto logDelete = [](Resource* r) {
        std::cout << " [CUSTOM DELETER] Freeing: "; r->use();
        delete r;
    };

    // unique_ptr<T, DeleterType>
    std::unique_ptr<Resource, decltype(logDelete)>
        up(new Resource("Eta", 7.0), logDelete);

    up->use();
    // logDelete called when up goes out of scope
}

```

```

std::cout << "\n";

// — 8. unique_ptr<T[]> – array ownership —————
std::cout << "-- 8. unique_ptr<T[]> for arrays --\n";
{
    auto arr = std::make_unique<int[]>(6);
    for (int i = 0; i < 6; i++) arr[i] = (i + 1) * 11;
    std::cout << "   Array: ";
    for (int i = 0; i < 6; i++) std::cout << arr[i] << " ";
    std::cout << "\n";
    // delete[] called automatically – NOT delete (that would be a bug)
}
std::cout << "\n";

// — 9. Factory function returning unique_ptr —————
std::cout << "-- 9. Factory function --\n";
{
    auto makeResource = [](const std::string& name, double val)
        -> std::unique_ptr<Resource> {
        return std::make_unique<Resource>(name, val);
    };

    auto r1 = makeResource("Factory-1", 11.0);
    auto r2 = makeResource("Factory-2", 22.0);
    r1->use();
    r2->use();

    // Convert unique_ptr to shared_ptr (valid – unique gives up ownership)
    std::shared_ptr<Resource> sp = std::move(r1);
    std::cout << "   Converted to shared_ptr: use_count="
        << sp.use_count() << "\n";
    sp->use();
}

std::cout << "\n=== unique_ptr demo complete ===\n\n";
}

// =====
// shared_ptr – Shared ownership via reference counting
// =====

void sharedPtrDemo() {
    std::cout << "=== shared_ptr: Shared Ownership ===\n\n";

```

```
// — 1. Basic shared ownership —————
std::cout << "-- 1. Basic shared_ptr and use_count --\n";
{
    auto sp1 = std::make_shared<Resource>("SharedAlpha", 100.0);
    std::cout << "  After sp1 created: use_count=" << sp1.use_count() << "\n";

    {
        auto sp2 = sp1;    // COPY – both share ownership
        std::cout << "  After sp2=sp1:  sp1.use_count()=" << sp1.use_count()
                    << "  sp2.use_count()=" << sp2.use_count() << "\n";
        sp2->use();

        auto sp3 = sp2;    // three owners
        std::cout << "  After sp3=sp2:  use_count=" << sp1.use_count() << "\n";
    }
    // sp2, sp3 go out of scope: count drops from 3 to 1
    std::cout << "  After sp2,sp3 out of scope: use_count=" << sp1.use_count() << "\n";
    sp1->use();
    // Resource alive – sp1 still holds it
}
// sp1 out of scope: use_count drops to 0 → Resource destroyed
std::cout << "\n";

// — 2. Shared across multiple containers —————
std::cout << "-- 2. Shared across multiple containers --\n";
{
    auto shared = std::make_shared<Resource>("CrossContainer", 200.0);

    std::vector<std::shared_ptr<Resource>> list1, list2, list3;
    list1.push_back(shared);    // count = 2
    list2.push_back(shared);    // count = 3
    list3.push_back(shared);    // count = 4

    std::cout << "  use_count with 3 containers: " << shared.use_count() << "\n";

    list1.clear();    // count = 3
    std::cout << "  After list1.clear(): " << shared.use_count() << "\n";

    list2.clear();    // count = 2
    std::cout << "  After list2.clear(): " << shared.use_count() << "\n";

    // shared itself still holds one reference
}
```

```

    // Resource won't die until 'shared' goes out of scope too
}
std::cout << "\n";

// — 3. reset() on shared_ptr —————
std::cout << "-- 3. reset() on shared_ptr --\n";
{
    auto sp1 = std::make_shared<Resource>("Beta-Shared", 300.0);
    auto sp2 = sp1;
    std::cout << "  use_count before sp1.reset(): " << sp1.use_count() << "\n";

    sp1.reset(); // sp1 releases its share; count drops to 1
    std::cout << "  sp1 after reset(): " << (sp1 ? "not null" : "null") << "\n";
    std::cout << "  sp2 use_count: " << sp2.use_count() << "\n";
    sp2->use(); // Resource still alive through sp2

    sp2.reset(); // last owner releases → Resource destroyed
    std::cout << "  sp2 after reset(): " << (sp2 ? "not null" : "null") << "\n";
}
std::cout << "\n";

// — 4. make_shared vs shared_ptr(new T) —————
std::cout << "-- 4. make_shared vs new (why make_shared is better) --\n";
{
    // Bad: two allocations (object + control block separately)
    //      also NOT exception-safe if bad_alloc happens between alloc and ctor
    std::shared_ptr<Resource> sp_bad(new Resource("NewStyle", 400.0));

    // Good: ONE allocation (object + control block together)
    //      exception-safe, better cache locality
    auto sp_good = std::make_shared<Resource>("MakeStyle", 500.0);

    std::cout << "  Both work; make_shared is always preferred.\n";
}
std::cout << "\n";

// — 5. Polymorphism with shared_ptr —————
std::cout << "-- 5. Polymorphism --\n";
{
    class Animal {
    public:
        virtual ~Animal() = default;
        virtual void speak() const = 0;
    };
}

```

```

        virtual std::string type() const = 0;
};
class Dog : public Animal {
public:
    void speak() const override { std::cout << " Woof!\n"; }
    std::string type() const override { return "Dog"; }
};
class Cat : public Animal {
public:
    void speak() const override { std::cout << " Meow!\n"; }
    std::string type() const override { return "Cat"; }
};

std::vector<std::shared_ptr<Animal>> animals;
animals.push_back(std::make_shared<Dog>());
animals.push_back(std::make_shared<Cat>());
animals.push_back(std::make_shared<Dog>());

for (const auto& a : animals) {
    std::cout << " [" << a.type() << "] ";
    a->speak();
}
std::cout << "\n";

// — 6. enable_shared_from_this —————
std::cout << "-- 6. enable_shared_from_this --\n";
{
    class SelfReferencing : public std::enable_shared_from_this<SelfReferencing> {
        std::string name;
    public:
        SelfReferencing(const std::string& n) : name(n) {
            std::cout << " SelfReferencing '" << name << "' created.\n";
        }
        ~SelfReferencing() {
            std::cout << " SelfReferencing '" << name << "' destroyed.\n";
        }

        std::shared_ptr<SelfReferencing> getSelf() {
            // shared_from_this() gives a shared_ptr to 'this'
            // WITHOUT creating a second control block
            return shared_from_this();
        }
    }
}

```

```

        void greet() const {
            std::cout << " Hello from " << name << "\n";
        }
};

auto sp = std::make_shared<SelfReferencing>("MySelf");
sp->greet();

auto sp2 = sp->getSelf();    // correct: shares control block
std::cout << " use_count after getSelf(): " << sp.use_count() << "\n";

// If you did: shared_ptr<SelfReferencing>(this) inside the class
// it would create a SECOND control block → double free → crash!
}
std::cout << "\n=== shared_ptr demo complete ===\n\n";
}

// =====
// weak_ptr – Non-owning observation
// =====

void weakPtrDemo() {
    std::cout << "=== weak_ptr: Non-Owning Observation ===\n\n";

    // — 1. Basic weak_ptr: observe without owning —————
    std::cout << "-- 1. Observe without owning --\n";
    {
        std::weak_ptr<Resource> wp;

        {
            auto sp = std::make_shared<Resource>("Observed", 1.0);
            wp = sp;    // weak assignment – does NOT increment use_count

            std::cout << " sp.use_count() after wp=sp: " << sp.use_count() << "\n";
            std::cout << " (use_count is still 1 – weak_ptr doesn't count)\n";

            // Must lock() to access safely
            if (auto locked = wp.lock()) {
                std::cout << " lock() succeeded: "; locked->use();
                std::cout << " use_count while locked: " << locked.use_count() << "\n";
            }
        }
    }
}

```



```

// sp out of scope: Resource destroyed (wp didn't prevent it)

std::cout << " After sp scope: wp.expired()="
           << std::boolalpha << wp.expired() << "\n";

// lock() after expiry returns empty shared_ptr
if (auto locked = wp.lock()) {
    std::cout << " lock() succeeded (should NOT happen)\n";
} else {
    std::cout << " lock() returned null – resource is gone.\n";
}
}
std::cout << "\n";

// — 2. The cycle problem and the fix —————
std::cout << "-- 2. Cycle: strong → memory leak --\n";
{
    struct NodeStrong {
        std::string name;
        std::shared_ptr<NodeStrong> next; // STRONG → causes cycle
        NodeStrong(const std::string& n) : name(n) {
            std::cout << " NodeStrong '" << name << "' created.\n";
        }
        ~NodeStrong() {
            std::cout << " NodeStrong '" << name << "' destroyed.\n";
        }
    };

    auto n1 = std::make_shared<NodeStrong>("A");
    auto n2 = std::make_shared<NodeStrong>("B");
    n1->next = n2; // n2 use_count = 2
    n2->next = n1; // n1 use_count = 2

    std::cout << " n1 use_count=" << n1.use_count()
              << " n2 use_count=" << n2.use_count() << "\n";
    // When scope ends: both use_counts drop to 1, NOT 0 → LEAK!
    std::cout << " (Expecting NO 'destroyed' messages – cycle prevents it)\n";
}
std::cout << " (Confirmed: destructors NOT called – memory leaked!)\n\n";

std::cout << "-- 3. Cycle fixed with weak_ptr --\n";
{
    struct NodeWeak {

```

```

std::string name;
std::weak_ptr<NodeWeak> next;    // WEAK → no cycle
NodeWeak(const std::string& n) : name(n) {
    std::cout << "  NodeWeak '" << name << "' created.\n";
}
~NodeWeak() {
    std::cout << "  NodeWeak '" << name << "' destroyed.\n";
}
};

auto n1 = std::make_shared<NodeWeak>("X");
auto n2 = std::make_shared<NodeWeak>("Y");
n1->next = n2;    // weak: n2 use_count stays 1
n2->next = n1;    // weak: n1 use_count stays 1

std::cout << "  n1 use_count=" << n1.use_count()
            << "  n2 use_count=" << n2.use_count() << "\n";
// scope ends: use_counts drop to 0 → BOTH destroyed!
}
std::cout << "  (Confirmed: both destroyed – no leak!)\n\n";

// — 4. Cache using weak_ptr —————
std::cout << "-- 4. Cache using weak_ptr --\n";
{
    // A cache that holds weak_ptr – doesn't prevent eviction
    std::vector<std::weak_ptr<Resource>> cache;

    auto loadResource = [&](const std::string& name, double val)
        -> std::shared_ptr<Resource> {
        // Check if cached version is still alive
        for (auto& wp : cache) {
            if (auto sp = wp.lock()) {
                if (sp->getName() == name) {
                    std::cout << "  Cache HIT: " << name << "\n";
                    return sp;
                }
            }
        }
        // Cache miss – create new and cache
        std::cout << "  Cache MISS: loading " << name << "\n";
        auto sp = std::make_shared<Resource>(name, val);
        cache.push_back(sp);    // weak reference
        return sp;
    };
}

```

```

};

// Load Resource-A → cache miss, created
auto r1 = loadResource("ResA", 1.0);
// Load Resource-A again → cache hit
auto r1b = loadResource("ResA", 1.0);

std::cout << "  r1 and r1b same? " << (r1.get() == r1b.get() ? "YES" : "NO") << "\n";

// r1 and r1b released – Resource-A evicted from memory
r1.reset();
r1b.reset();
std::cout << "  ResA released.\n";

// Load Resource-A again → cache miss (weak expired)
auto r1c = loadResource("ResA", 1.5);
r1c->use();
}
std::cout << "\n";

// — 5. Observer pattern with automatic cleanup —————
std::cout << "-- 5. Observer pattern --\n";
{
    class EventBus {
        std::vector<std::weak_ptr<std::string>> listeners;
    public:
        void subscribe(std::weak_ptr<std::string> listener) {
            listeners.push_back(listener);
        }

        void publish(const std::string& event) {
            std::cout << "  Publishing: " << event << "\n";

            // Remove expired listeners and notify live ones
            auto it = listeners.begin();
            while (it != listeners.end()) {
                if (auto sp = it->lock()) {
                    std::cout << "    → " << *sp << " received: " << event << "\n";
                    ++it;
                } else {
                    it = listeners.erase(it); // silently remove expired
                }
            }
        }
    }
}

```

```

    }

    int listenerCount() const {
        return static_cast<int>(
            std::count_if(listeners.begin(), listeners.end(),
                [](const auto& wp){ return !wp.expired(); }));
    }
};

EventBus bus;
auto l1 = std::make_shared<std::string>("ListenerA");
auto l2 = std::make_shared<std::string>("ListenerB");
auto l3 = std::make_shared<std::string>("ListenerC");

bus.subscribe(l1);
bus.subscribe(l2);
bus.subscribe(l3);

bus.publish("AppStarted");
std::cout << "  Active listeners: " << bus.listenerCount() << "\n";

// ListenerB unsubscribes by going out of scope
l2.reset();
std::cout << "\n  (ListenerB released)\n";

bus.publish("DataUpdated"); // Only A and C receive it
std::cout << "  Active listeners: " << bus.listenerCount() << "\n";
}

std::cout << "\n=== weak_ptr demo complete ===\n\n";
}

```

```

// =====
// COMPARISON AND GUIDELINES
// =====

```

```

void smartPtrGuidelines() {
    std::cout << "=== Smart Pointer Selection Guide ===\n\n";
    std::cout << R"(
DECISION FLOWCHART:

```

Q: Do you need ownership (object's lifetime depends on this pointer)?

|

└─ NO: Use a raw pointer (T*) or reference (T&) – no ownership.

|

└─ YES:

Q: Is ownership EXCLUSIVE (only one owner at a time)?

|

└─ YES → unique_ptr<T>

|

- Zero overhead vs raw pointer
- Explicit move required (std::move)
- Factory functions return unique_ptr

|

└─ NO → shared ownership?

|

└─ YES → shared_ptr<T>

|

- Ref-counted – object lives until last owner dies
- Atomic ref count: ~3-5x slower than unique_ptr
- Use make_shared<T> (not new T)

|

└─ OBSERVE only (no ownership, break cycles)?

→ weak_ptr<T>

- Never prevents destruction
- Must lock() before dereferencing
- Cycles: use weak_ptr for back-pointers

PERFORMANCE (relative cost per operation, approximate):

unique_ptr	Same as raw pointer – compiler inlines it
shared_ptr	~3-5x raw pointer (atomic ref count ops)
weak_ptr	lock() slightly more than shared_ptr

COMMON MISTAKES TO AVOID:

✗ unique_ptr<T> p2 = p1; → Compile error (use std::move)

✗ shared_ptr<T> sp(raw_ptr);

shared_ptr<T> sp2(raw_ptr); → TWO control blocks → double free!

✗ class T { void f() {

shared_ptr<T> me(this); // Second control block → crash!

}}; // Fix: inherit enable_shared_from_this

✗ shared_ptr<T> sp1 → sp2 → sp1; → Cycle → leak! Use weak_ptr.

✗ *wp (weak_ptr dereference) → Must call wp.lock() first!

✗ if(wp.expired()) use(wp); → TOCTOU race. Always lock() instead.

);

std::cout << "\n=== Guidelines complete ===\n\n";

}

```
// =====  
//  MAIN  
// =====  
  
int main() {  
    std::cout << "]\n";  
    std::cout << " | Smart Pointers – Complete Demo |]\n";  
    std::cout << "]\n\n";  
  
    autoPtrHistory();  
    uniquePtrDemo();  
    sharedPtrDemo();  
    weakPtrDemo();  
    smartPtrGuidelines();  
  
    std::cout << "]\n";  
    std::cout << " | All demos complete. |]\n";  
    std::cout << "]\n";  
    return 0;  
}
```

Quick Reference — Chapters 18–21

CHAPTERS 18–21 CHEAT SHEET

FILE I/O

ofstream → write only ifstream → read only
fstream → read + write
seekg(n, dir) → move read ptr tellg() → get read pos
seekp(n, dir) → move write ptr tellp() → get write pos
dir: ios::beg / ios::cur / ios::end

EXCEPTIONS

throw obj; try { } catch(const T& e) { } catch(...) { }
throw; → re-throw current exception (preserves type)
std::set_terminate(fn) → called when nothing catches
noexcept → promises no throw (terminate if it does)
exception_ptr → store/transport exceptions across contexts

NAMESPACES

namespace ns { } → define
ns::name → qualified access
namespace alias = ns → short synonym
using ns::name → import one name
using namespace ns → import all (use locally only)
namespace { } → file-private (anonymous)

SMART POINTERS

unique_ptr → exclusive, zero overhead, move-only
shared_ptr → shared, ref-counted, copyable
weak_ptr → non-owning, must lock(), breaks cycles
make_unique<T>(args) → preferred construction
make_shared<T>(args) → preferred construction
wp.lock() → returns shared_ptr or null
wp.expired() → true if no owners remain

Complete code examples — using `iostream` throughout